Final Project - Machine Learning

Analysing NBA Players Throughout the Years

Angelo Miculescu

1. Introduction

With the NBA playoff finals underway, I thought it suitable to search for a dataset and attempt to make predictions related to the league and its players. The main purpose of this project will be to observe whether a suitable model can be built that can predict an NBA player's position based on several features of said player.

Along the way, several other observations will be made that would be interesting to any NBA or sports fan in general. These will be discussed in the data visualization section. The machine learning insights (including the plots created from its output will be in the later Machine Learning section).

2. Data

My dataset consists of a single source of data downloaded from Kaggle. The data consists of (nearly) every NBA player to ever play in the league, their career stats, the team they played with in their prime, their college and draft round and number, among many other columns.

For the purposes of this project, only a sample of these columns were selected to be used as features for predictions. These include: PTS (points), REB (rebounds), AST (assists), HEIGHT & WEIGHT. Bar a few exceptions (DRAFT_YEAR, DRAFT_ROUND etc.) the majority of other fields were categorical and would be of little use in the predictions. In the case of these columns, some were used in some earlier visualisations as they give rise to some interesting observations.

3. Importing

The data was simply downloaded from the Kaggle website using a link provided on the page featuring the data. This was then stored in a location in my directory, from where it was taken and read into the DataFrame known as bball_df.

```
# Download latest version of the dataset from Kaggle
path =
kagglehub.dataset_download("yagizfiratt/nba-players-database")

# List the files in directory, use file name obtained below to
create DataFrame
print(os.listdir(path))

bball_df = pd.read_csv(path + '/PlayerIndex_nba_stats.csv')
```

It is worth noting that unlike other projects I have carried out, this one does not seek to combine datasets but instead sticks to a single one. This was largely due to the size of the dataset. Under normal circumstances, ~4000 rows and 29 columns would not be significant. However, in this case, those 4000 rows happened to each represent a distinct NBA player and finding another dataset that has collected different information for each player was a difficult task.

4. Data Preparation & Pre-processing

Many steps were taken to ensure the data was adequate enough to gain insight from, both from a basic visualization perspective as well as Machine Learning. The first task was to cover up all missing values from the fields to be used as features in training models. This involved the following:

- PTS, AST and REB - Through quick observation, it was determined that none of the players missing stats were of high significance in the world of basketball and therefore none would likely average more than the mean of the entire dataset. As such, each player missing a stat was given a random number between 0 and the mean average of that stat in the dataset (in this case it was points and the mean for the dataset was 7).

```
bball_df.loc[bball_df['PTS'].isna(), 'PTS'] =
np.random.randint(0, 701, size=bball_df['PTS'].isna().sum())
/ 100.0
```

- HEIGHT, WEIGHT, POSITION - instances of missing height, weight or position were removed from the dataset.

```
bball_df.dropna(subset=['HEIGHT'], inplace=True)
```

The second task involved converting certain fields to a more usable form. Take for example the following:

- POSITION - In order to simplify the task of predicting positions, it was decided to convert all mixed positions into one of the three main ones: Guard, Forward & Center (in this case, players whose positions was Center/Forward were simply given the position Center).

```
bball_df.loc[bball_df['POSITION']=='C-F', 'POSITION'] = 'C'
```

- HEIGHT - This field was naturally selectable as an object in the form “F-I”, where F is feet and I is inches. The function below split these up, converted them to ints and added them together as inches only.

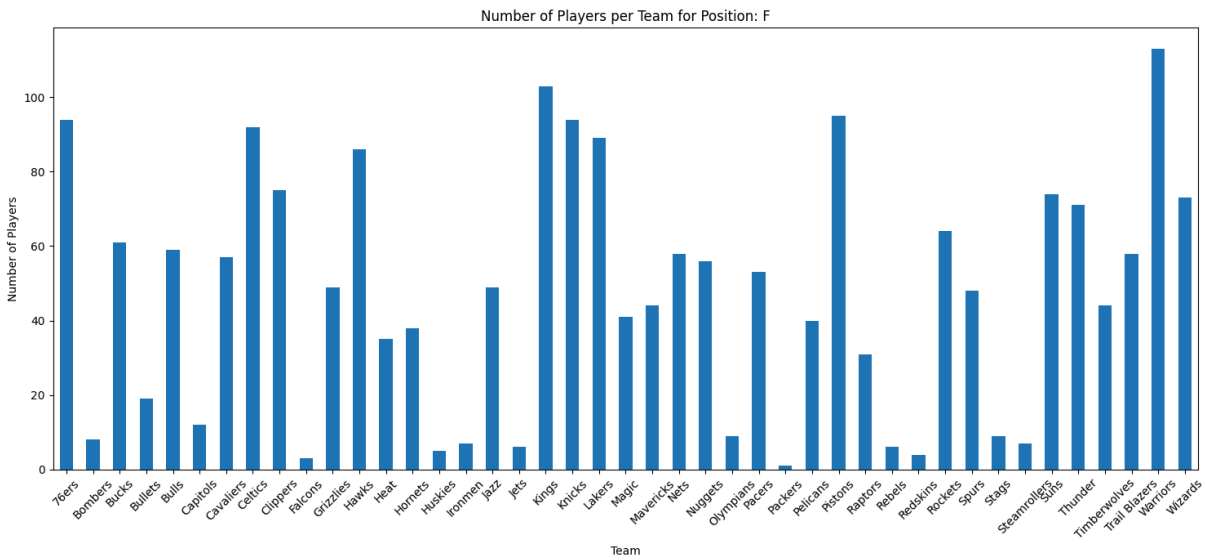
```
def convert_height(height):
  return int(height.split('-')[0])*12 +
int(height.split('-')[1])
```

Lastly, for the purposes of some of the predictions later on, a separate dataframe was created which included only the players who had at least one main stat within the 90th percentile of the total group (we will call this the 90th percentile dataset from now on).

```
players_90th_percentile_df = bball_df[
    (bball_df['PTS'] >= percentiles_90['PTS']) |
    (bball_df['REB'] >= percentiles_90['REB']) |
    (bball_df['AST'] >= percentiles_90['AST'])
].copy()
```

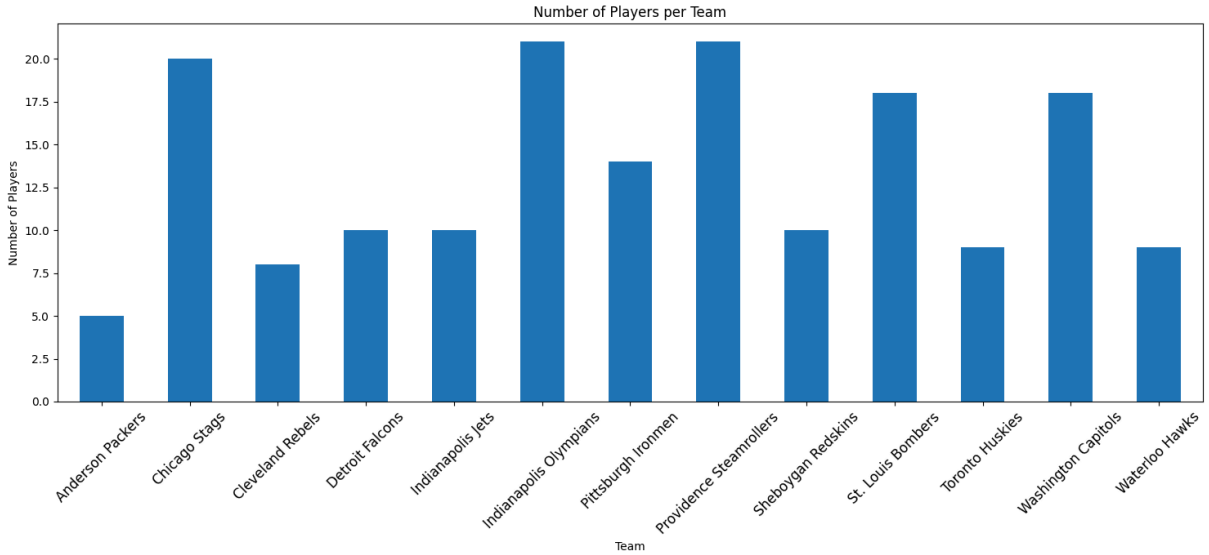
5. Data Visualisation & Insights

The first of a few visualisations created was that of the number of players per team for each position in the dataset (other position graphs are available in the Notebook provided):



This seemingly simple graph gave rise to an amusing observation. While it is clear to see some of the mainstream teams garnering many players over the years, this graph also clearly highlights several teams that haven't, teams that were, perhaps, never part of the NBA and were only around for its predecessors, the National Basketball League (NBL) or Basketball Association of America (BAA). A separate graph was printed to illustrate this finding.

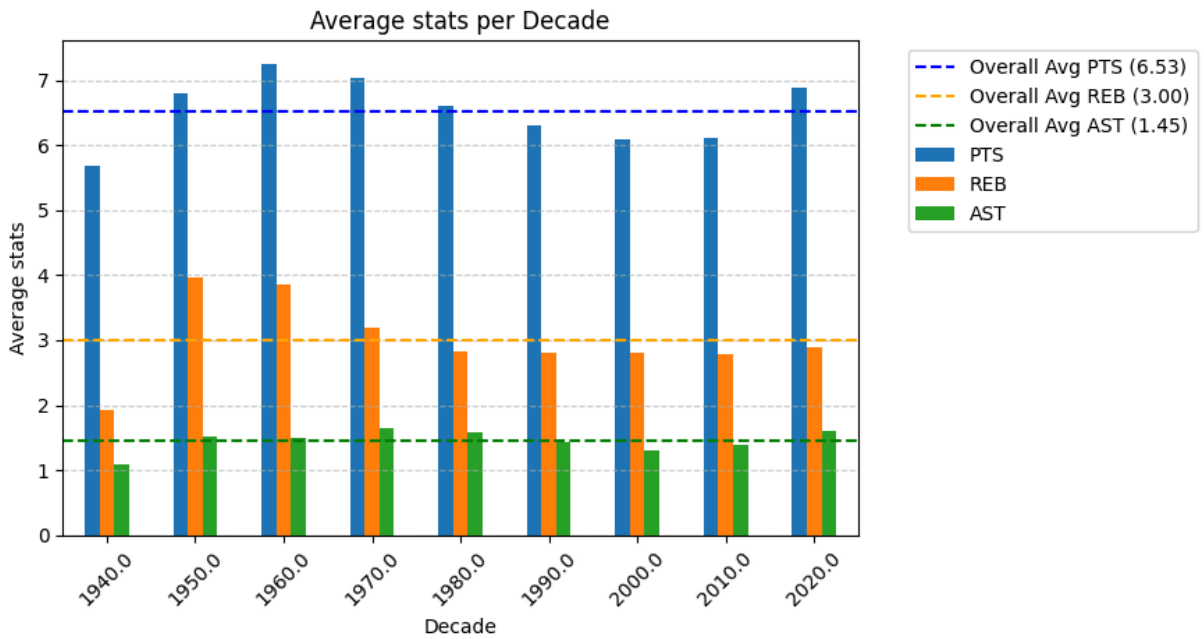
NB - Each player was assigned only one team, that which he played his prime years on, otherwise the number per team would be much higher.



A very simple observation but one that would intrigue even the most dedicated fans of the sport and league, myself included.

The next graph created was done so for the purposes of settling a debate. This debate has been constantly raging amongst fans of basketball, young and old: which era of basketball was best; which was tougher, which was faster, which defended better, which scored better.

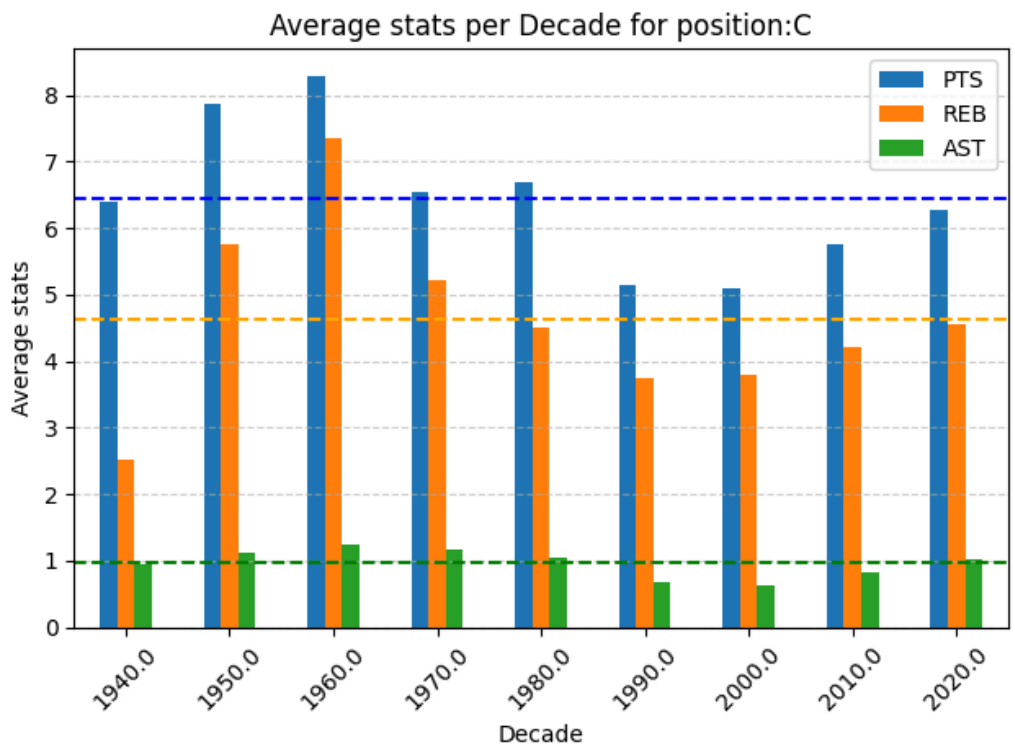
Unfortunately, given the lack of information in the dataset, one can't answer all of these questions, but a few assertions can be made:



For example, by analysing the average stats per decade and comparing them, as well as mapping out the actual mean of the whole dataset, we can come to certain conclusions:

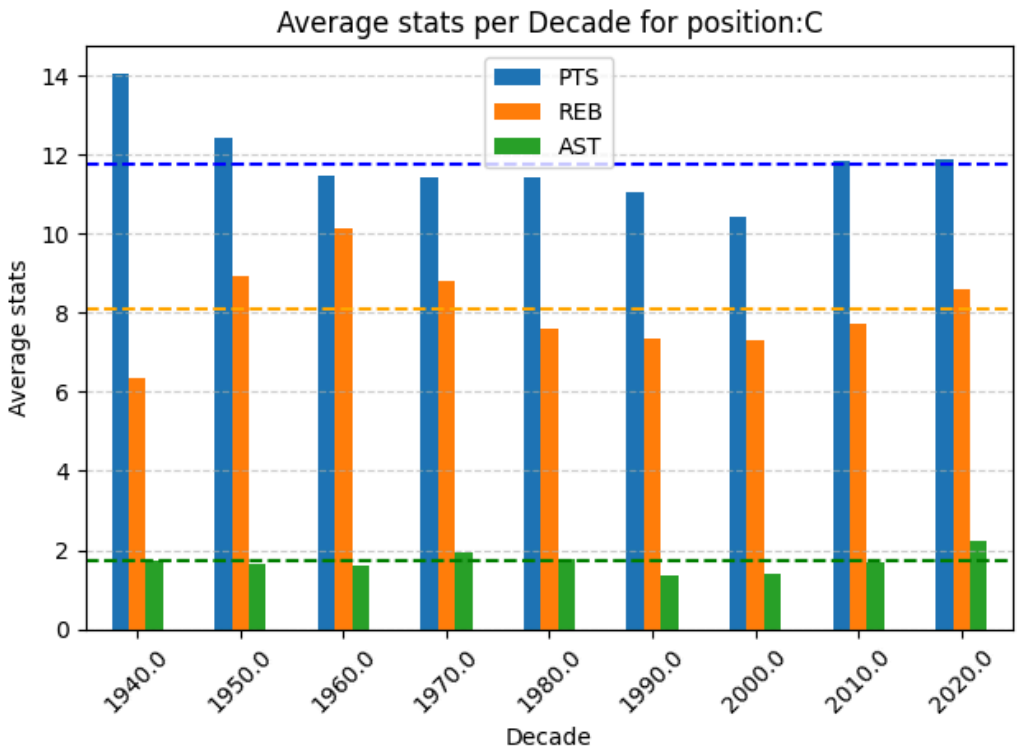
1. Points were low in the 40's, indicating a low level of play, natural for a league which was just starting up (the game of basketball only having only been recently founded in 1891).
2. This was followed by three decades of some of the highest scoring ever, while the defence was still lacking. For basketball fans, this era is known for housing names such as Wilt Chamberlain and Kareem Abdul-Jabbar, players known for scoring an absurd amount of points.
3. Following this we have multiple decades of lower scoring, an indication of the rise in defensive capabilities and acknowledgement. Players such as Dikembe Mutombo and Michael Jordan are famous for their dedication to the defensive end.
4. We then see a dramatic rise in points in the most recent decade, perhaps due to the rise in popularity of the 3 pointer, which almost all players in the league now can use very efficiently, although more information would be required to confirm this.
5. Assists and Rebounds don't see much change over the years. The 40's show the lowest averages, again due to the lack of experience. The 50's and 60's show the highest averages, likely due to players like Wilt and Kareem showing dominance (note also that due to the smaller size of the league, higher averages from superstars such as these hold greater weight in a graph such as this.)

The graph above was replicated for each individual position, the most interesting of which, was the Centers:



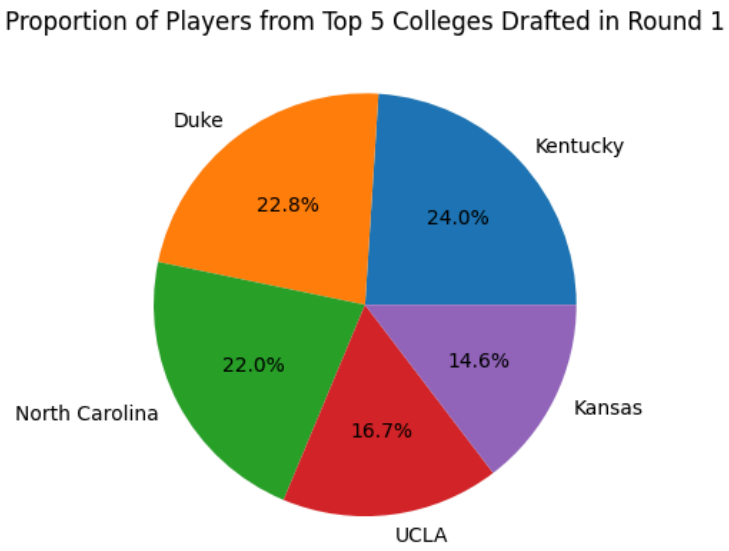
The dominance of the giants in the 50's and 60's cannot be overstated, as their averages far surpass those of other eras.

An interesting contradictions occurs when recreating this graph but using the 90th percentile dataset:

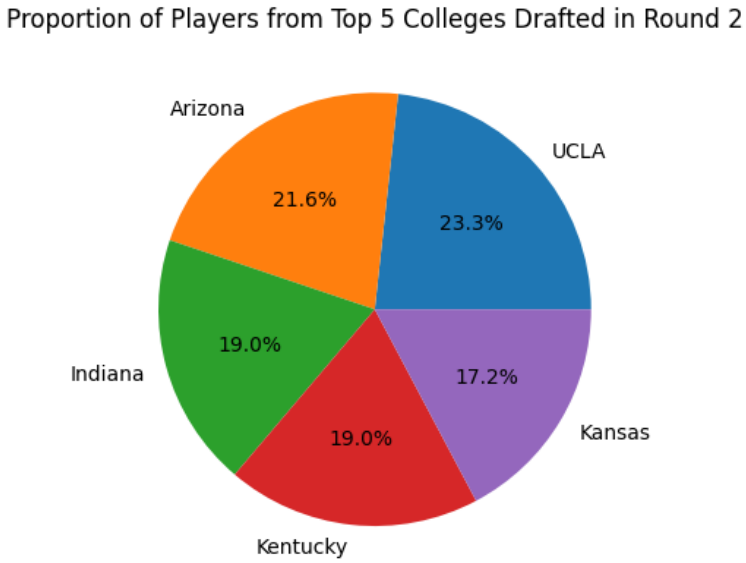


The 40's are surpassing every other decade in points. Upon investigation, this was due to the dataset only containing a mere 20 players at the Center position, of which only 5 averaged less than 10 points. This lack of total players plays a huge factor in increasing the averages like we see above.

The last two visualisations created were done to compare the number of players each college in the dataset has sent to the NBA, divided by round. For context, each year there is an NBA Draft where players from college and abroad are selected by teams to play on their team. There are 2 rounds consisting of 30 picks:



We can see that Kentucky is the most popular college for first round picks, followed by Duke and North Carolina.



On the other end, it seems like UCLA is most popular for second round picks, followed by Arizona. We do also see Kentucky and Kansas reappearing for the second round, making them the overall best colleges to play for if you are looking to make the NBA.

6. Machine Learning

To reiterate, the purpose of this project is to develop a model to predict the position of a player based on his stats and physical attributes. The initial attempt involved using a voting classifier which was to take the answers from four base classifiers (see below), taking the most popular. In this case, voting was set to soft, meaning it took probabilities rather than

distinct outcomes (please note, constant random state was chosen such that the predictions could be more consistently measured).

```
lr = LogisticRegression(random_state=42)
rf = RandomForestClassifier(random_state=42)
svc = SVC(probability=True, random_state=42)
gb = gb_clf = GradientBoostingClassifier(random_state=42)

vc = VotingClassifier(estimators=[('lr', lr), ('rf', rf), ('svc',
svc), ('gb', gb)], voting='soft')
```

It's important to note that the following steps were taken during this and most other model creation stages:

- The features and target variable labels were defined and passed through the datasets to create the feature and target dataframes.

```
features = ['PTS', 'REB', 'AST']
target = 'POSITION'
X = bball_df[features]
y = bball_df[target]
```

- These were then passed through a train-test split function to be used to train and test the model above:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

- Last important step involves normalising the data such that no individual feature holds greater weight than another (in this case this was done within a pipeline which held the StandardScaler as well as the voting classifier above:

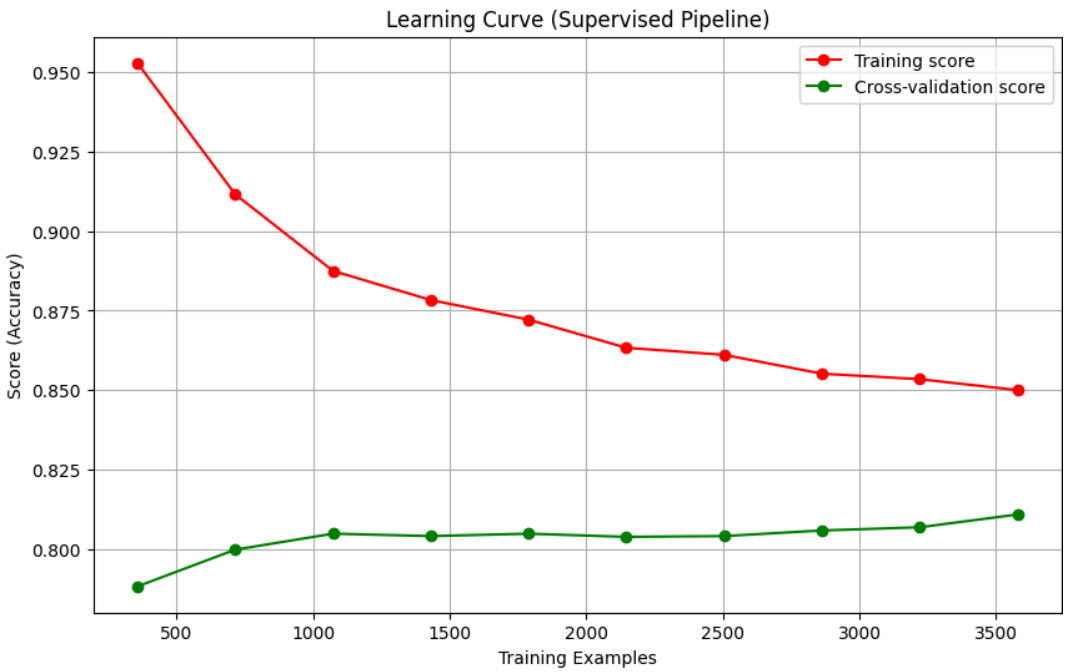
```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('voting', vc)
])
```

The initial test accuracy read ~0.7183 which was good but could be improved upon. To increase the accuracy, we first decided to add height and weight to the features to help better distinguish between Centers and Forwards:

```
New_features = ['PTS', 'REB', 'AST','NEW_HEIGHT','WEIGHT']

New_X = bball_df[New_features]
```

This resulted in a better testing accuracy of ~82%.

The next step was to plot a learning curve to investigate whether this model was experiencing any under- or over-fitting:



As we can see, running over 10 distinct training sizes, the last being 3500 in size, no convergence occurs between the training score and test score. Even though the training score lowers greatly from near 100% to 85%, it is still larger than the highest test score, clocking in at ~81.7%.

As such, it was necessary to take measures to reduce this over-fitting. This came in the form of hyperparameter tuning through randomized grid search cross validation. Please note that through testing, it was determined that the gradient boosting model within the voting classifier was not only increasing run-time but also decreasing accuracy. It was therefore removed.

The parameter distribution list used to change the hyperparameters looks as follows:

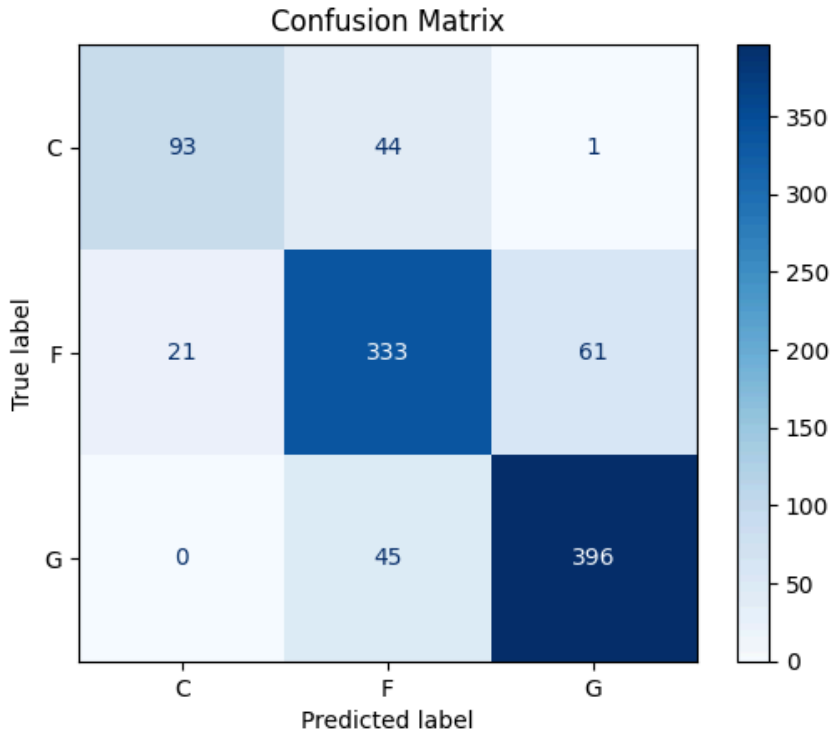
```
param_distributions = {

    'voting__lr__C': uniform(loc=0.01, scale=1), # Selecting
randomly from a continuous distribution

    'voting__rf__n_estimators': randint(50, 200), # Selecting
randomly from an integer distribution

    'voting__rf__max_depth': [35, 25],

    'voting__svc__C': uniform(loc=0.01, scale=1),

    'voting__svc__gamma': ['scale', 'auto'],

    #'voting__gb__n_estimators': randint(50, 200),

    #'voting__gb__learning_rate': uniform(loc=0.01, scale=0.2),
```

```
}
```

Here, uniform(loc=0.01, scale=1) randomly selects a value between 0.01 and 1.01 & randint(50, 200) selects a random integer between 50 & 200. For run-time efficiency, cv=3 was used. The best model yielded a result of 0.8269617706237424 or ~82.6%, an improvement from before.

Testing the best model on the training set, we saw a similar score of ~82.9%, demonstrating that overfitting had indeed been resolved.

Accuracy isn't the only form of measurement however as we also calculated a confusion matrix as well as a classification report displaying our f1-score:

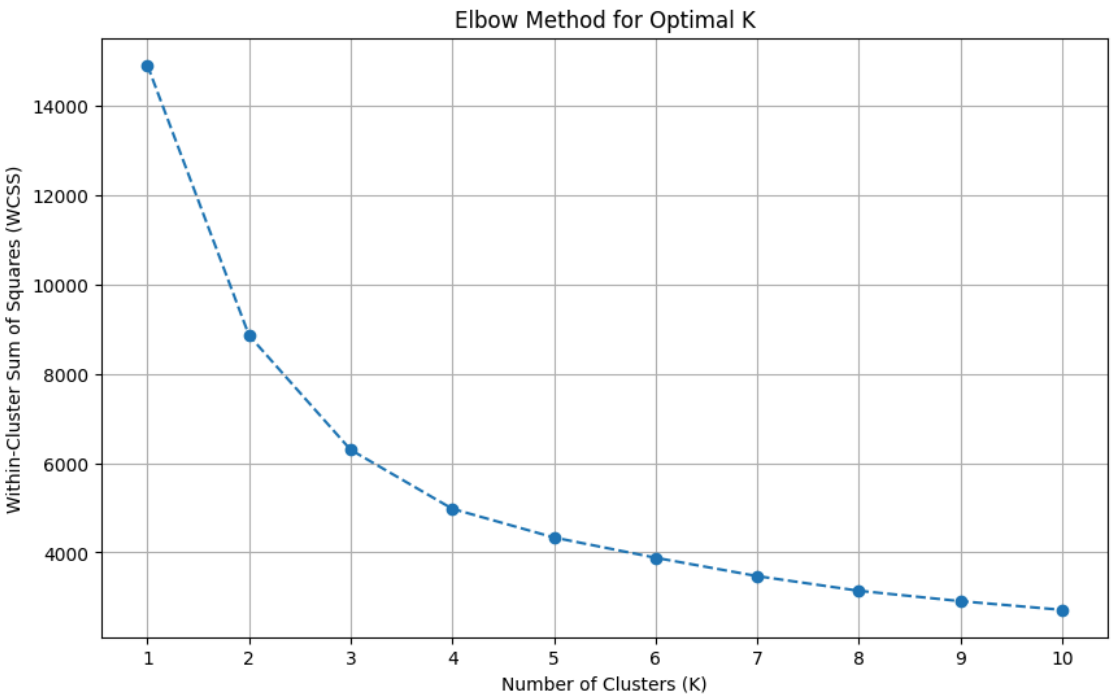



```
Classification Report:
              precision    recall  f1-score   support

           C       0.75      0.72      0.74       138
           F       0.78      0.79      0.79       415
           G       0.88      0.88      0.88       441

    accuracy                           0.82       994
   macro avg       0.80      0.80      0.80       994
weighted avg       0.82      0.82      0.82       994
```

From these two reports, we can clearly notice the best precision was found for predicting Guards, with the worst being Centers, with Forwards just slightly better off. This is likely to the HEIGHT and AST features making guards more differentiable as they would average the smallest values for height and largest values for assists.

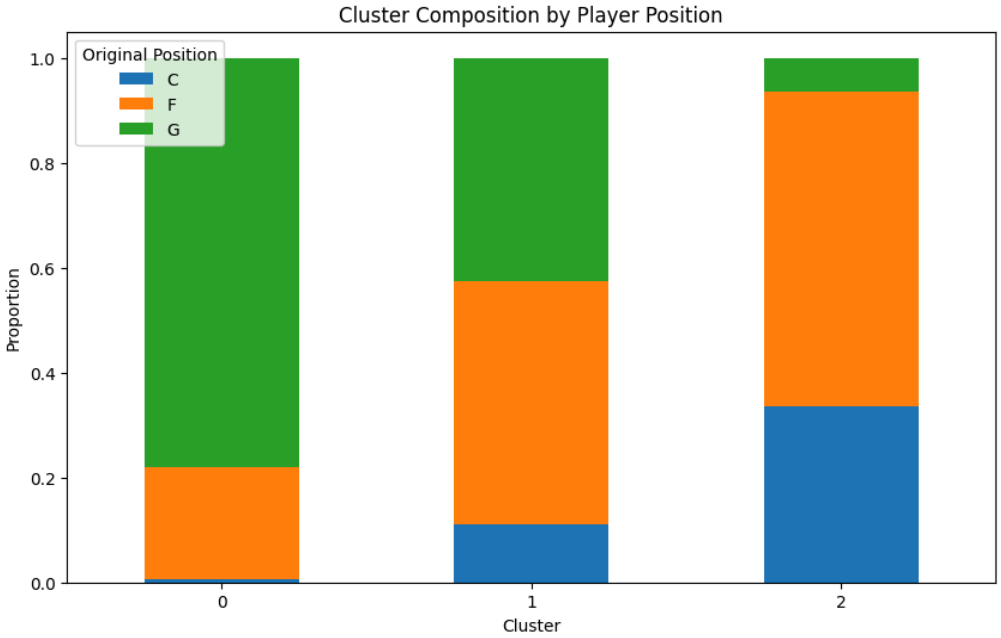
One important thing to note however is the support column, particularly for Centers. Support lists the number of that particular observable were present in the dataset. In this case, C has 3 times less representation than the other two and thus less opportunity to raise its precision with more predictions. This isn't a given however. A second thing to note is the average stats for the different players (found above in the visualisation section). Due to the high number of weak players at each position, this brings averages down across the board, making them look very similar to one another. This might be what is getting a lot of C being classified as F and vice versa.

It was decided to take an unsupervised approach in order to evaluate whether it was possible for a machine to predict a players position based on stats without being given an answer to train on. Kmeans clustering seemed like the best choice for a classifier here. The first objective when using kmeans is to find out how many clusters we should be checking for (although we would be looking for 3 clusters anyway, this seemed like an appropriate first step regardless):

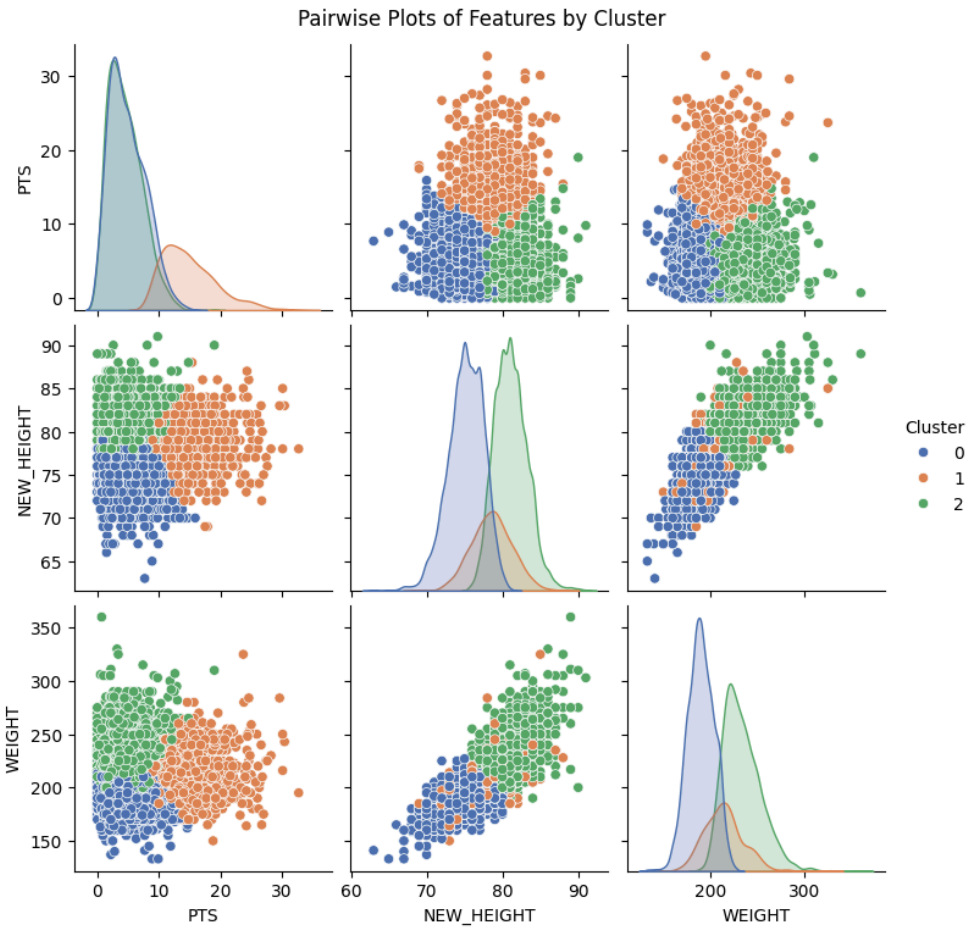


We notice a decline in the gradient after k=2 (once again due to the weaker players pulling down the averages, one could assume the 2 clusters here would be tall players vs short players). Nevertheless, 3 clusters were chosen as we require 3 positions to be filled.

After using a variety of features and even implementing feature engineering to create new, potentially useful features, the best results we could get are shown below in the plotted pairplots and barchart:

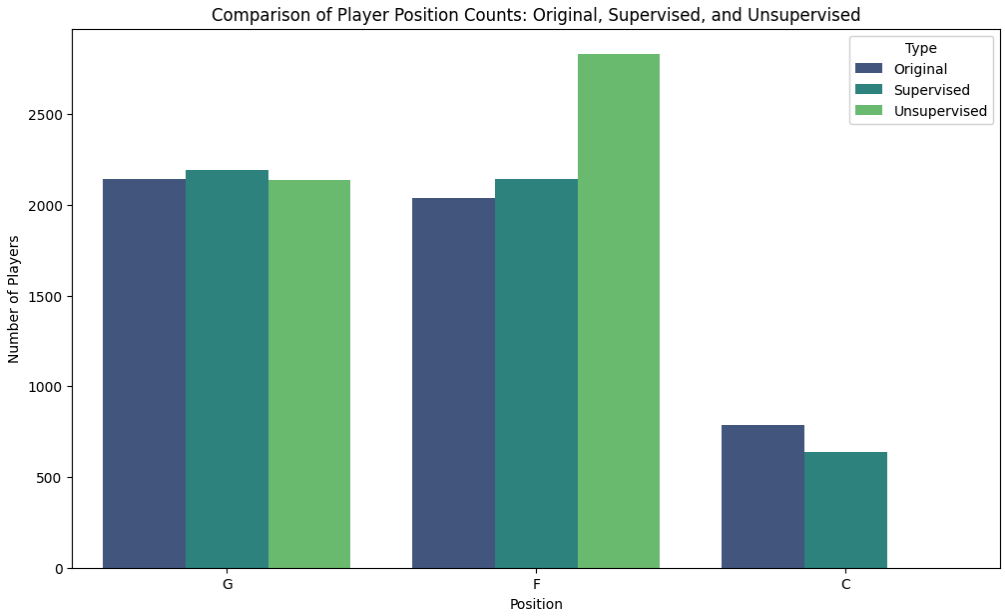


Here, we run into another issue, in that the clusters were being created in equal sizes. This means we're trying to compute 3 times the amount of centers we should have, and since we don't have that many, the clusters predicting Centers (number 2 above & below), that cluster is being dominated by Forwards.



We see from the pair plots that the right approach was taken by the model in a few of the cases. Take for example, points vs height, we have tall players on the right, small on the left and the forwards (who would also typically score more points) in the centre. This isn't taking into account however, tall or short players who also score a lot or forwards who don't score a lot.

A final plot was created showing the difference between the different model predictions:



As previously discussed, supervised mixed up certain F's & C's but otherwise was accurate. Our unsupervised model didn't predict any Centers in the end, all of them falling under F due to the large amount of that position in the cluster. This is due to the method of calculation of the position:

```
cluster_composition =
players_90th_percentile_df.groupby('Cluster')['POSITION'].value_c
ounts(normalize=True).unstack(fill_value=0)

cluster_to_position_mapping =
cluster_composition.idxmax(axis=1).to_dict()

players_90th_percentile_df['Predicted_Position_Cluster'] =
players_90th_percentile_df['Cluster'].map(cluster_to_position_map
ping)
```

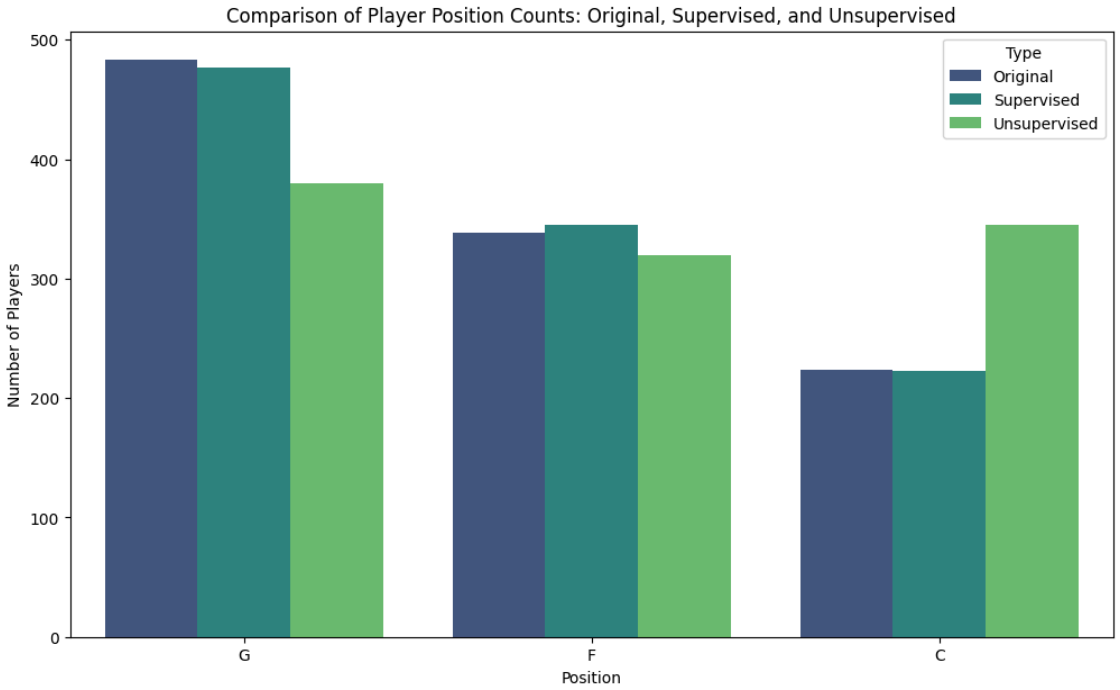
Breaking it down into 3 components:

1. the positions in the DF are grouped by the various clusters created by the kmeans model.

2. The most frequent positions in each cluster are mapped into a dictionary mapping position to cluster.
3. This mapping is then used to add the new predicted position as a new column to the DataFrame.

Due to point 2 using frequency to calculate the new position, cluster 2 (which contains most C's) players are mapped as Forwards. This further emphasises the points made above in the supervised predictions section.

A similar activity was undertaken for the 90th percentile dataset, for both the supervised and unsupervised model (below is the final result, other results can be found within notebook):



As we can see, Centers are better predicted this time around. Our previous issue of equally sized clusters still occurs however, as now we can see many forwards added to the Centers cluster and similarly for the Guards into the Forwards cluster. The supervised model has also seen improvement, as the new efficiency, before hyperparameter tuning is ~83.7%.

7. Conclusion

2 distinct models were built in order to predict the positions of players within the NBA. One of these models (supervised) did an acceptable job of this, as it was able to spot patterns within the training set that allowed it to adapt to the test set. This is all the more obvious when it was run on the 90th percentile group, where Centers were almost totally predicted, with minor differences arising between Guards and Forwards. One detail this puts into perspective is the disparity between the few star calibered players in the league who distinguish themselves as being the best in their position, versus the vast amount who's stats

fall hidden in the crowd of other players. Ideally, we would have access to other scoring metrics that could've been used to better distinguish these players by their position.

Our final results also clearly outlined some of the limitations surrounding unsupervised learning, with the clusters created being all of even size. In future, one adjustment to make would be to create dummy rows for the Center position to have the total number be on par with the other two. These could then be removed once new positions were determined by the model, potentially leading to much more accurate results.

8. Appendix

- Main data source - https://www.kaggle.com/datasets/yagizfiratt/nba-players-database
- Setting up Kaggle API - https://www.kaggle.com/docs/api#authentication